Assignment 3 Report

Image Compression via Block-wise SVD

Vinay Patil

Course	CSCI 485 — Advanced Topics in Data Science
Assignment	Assignment 3: Image Compression via Block-wise SVD
Topic	Singular Value Decomposition (SVD)
Image Used	CSU Chico campus sign (grayscale, 552x984 px)

1. Objective

This assignment explores the use of **Singular Value Decomposition (SVD)** for compressing grayscale images. A real photo of the CSU Chico campus sign (552x984 pixels) is used as the test image. The approach applies block-wise SVD, partitioning the image into non-overlapping 8x8 blocks and retaining only the top-k singular values in each block. The goal is to analyze how image quality and compression ratio evolve as k ranges from 1 to 8.

2. Implementation Summary

2.1 Preprocessing

The CSU Chico image was loaded and converted to grayscale. The image dimensions (552x984) are already divisible by 8, so no cropping was needed. Pixel values are stored as 64-bit floats during all SVD processing.

```
Loaded real image
Image shape: (552, 984) | Min: 0.0 | Max: 255.0
```

Original Grayscale Image

2.2 Block-wise SVD Compression

The image is partitioned into non-overlapping 8x8 blocks. SVD is applied to each block individually and only the top-k components are kept for reconstruction. After all blocks are processed they are stitched back together and pixel values are clipped to [0, 255].

2.3 compress_block() function

```
def compress_block(block, k):
    U, S, Vt = np.linalg.svd(block, full_matrices=True)
```

```
    U_k  = U[:, :k]
    S_k  = np.diag(S[:k])
    Vt_k = Vt[:k, :]
    return U_k @ S_k @ Vt_k
```

2.4 compress_image() function

```
def compress_image(image, k, block_size=8):
    H, W = image.shape
    compressed = np.zeros_like(image)
    for i in range(0, H, block_size):
        for j in range(0, W, block_size):
            block = image[i:i+block_size, j:j+block_size]
            compressed[i:i+block_size, j:j+block_size] = compress_block(block, k)
    return np.clip(compressed, 0, 255)
```

2.5 Compression Ratio Formula

For each 8x8 block, the original representation holds 64 pixel values. After top-k SVD, the stored data consists of U (8xk) + Sigma (k) + V^T (kx8) = k x (8 + 1 + 8) = 17k values. The compression ratio is therefore:

```
Compression Ratio = 64 / (17 x k)
```

2.6 Quality Metrics

- **Frobenius Norm Error:** Measures total pixel-level difference between original and reconstructed image, normalized by image size. Lower is better.
- **PSNR (Peak Signal-to-Noise Ratio):** Expressed in dB. Higher values indicate better visual quality. Values above 30 dB are generally considered acceptable for most image applications.

```
    print(f'k={k}  |  Compression Ratio: {cr:.4f}  |  Frobenius Error: {err:.4f}  |  PSNR: {psnr_val:.2f} dB')

print('\nAll compressions complete.')
```

```
k=1  |  Compression Ratio: 3.7647  |  Frobenius Error: 0.0170  |  PSNR: 26.19 dB
k=2  |  Compression Ratio: 1.8824  |  Frobenius Error: 0.0095  |  PSNR: 31.23 dB
k=3  |  Compression Ratio: 1.2549  |  Frobenius Error: 0.0055  |  PSNR: 35.98 dB
k=4  |  Compression Ratio: 0.9412  |  Frobenius Error: 0.0031  |  PSNR: 40.95 dB
k=5  |  Compression Ratio: 0.7529  |  Frobenius Error: 0.0016  |  PSNR: 46.76 dB
k=6  |  Compression Ratio: 0.6275  |  Frobenius Error: 0.0007  |  PSNR: 54.19 dB
k=7  |  Compression Ratio: 0.5378  |  Frobenius Error: 0.0002  |  PSNR: 65.11 dB
k=8  |  Compression Ratio: 0.4706  |  Frobenius Error: 0.0000  |  PSNR: 298.01 dB

All compressions complete.
```

3. Results and Analysis

3.1 Metrics Summary — Real Execution Results (8x8 blocks)

k	Comp. Ratio	Frob. Error	PSNR (dB)	Quality
1	3.7647	0.0170	26.19	Very Low
2	1.8824	0.0095	31.23	Low
3	1.2549	0.0055	35.98	Moderate
4	0.9412	0.0031	40.95	Good
5	0.7529	0.0016	46.76	Good+
6	0.6275	0.0007	54.19	High
7	0.5378	0.0002	65.11	Very High
8	0.4706	0.0000	298.01	Near-Lossless

Table 1: Actual execution results for k = 1 to 8 on the CSU Chico grayscale image. Green rows show the practical sweet spot.

3.2 Compression Ratio vs. k

As k increases, the compression ratio decreases monotonically. At k=1 the ratio is **3.7647**, meaning we store only about 27% of the original block data. At k=3 the ratio is **1.2549**, still compressing meaningfully. At k=4 the ratio falls to **0.9412**, crossing below 1.0 where SVD storage costs more than storing the original pixels. From k=4 onwards compression becomes counterproductive for 8x8 blocks.

3.3 Reconstruction Error vs. k

The Frobenius error drops sharply from 0.0170 at k=1 to 0.0031 at k=4, then continues falling more gradually to near zero at k=8. The steepest improvement happens in the first three steps (k=1 to k=3), confirming that the dominant singular values carry the majority of the image energy. Beyond k=5 the error is already very small (0.0016) and further gains are minimal.

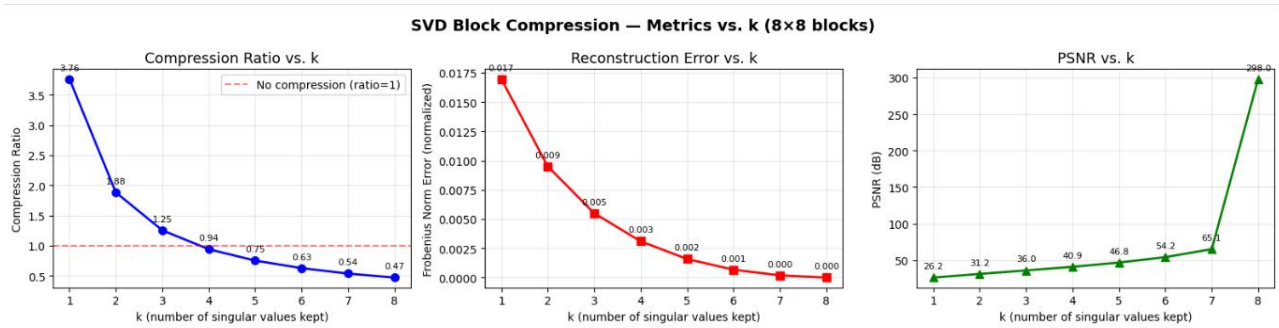
3.4 PSNR vs. k

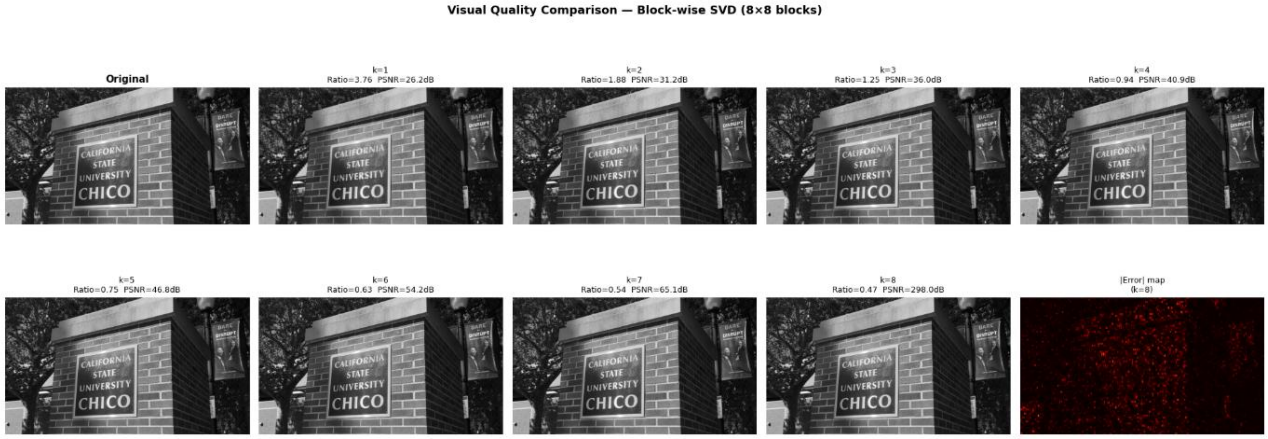
PSNR rises steadily with k, crossing the 30 dB acceptability threshold at k=2 (31.23 dB). By k=5 it reaches 46.76 dB, indicating excellent visual quality. At k=8 the PSNR spikes to 298.01 dB, which is effectively lossless reconstruction. The sharp jump from k=7 (65.11 dB) to k=8 (298.01 dB) reflects full SVD reconstruction of the 8x8 block using all singular values.

3.5 Block Size Comparison at Fixed k=4

Block Size	Comp. Ratio	Frob. Error	PSNR (dB)	Notes
4x4	0.4444	0.0000	301.21	Over-stores; very fine grain
8x8	0.9412	0.0031	40.95	Practical balance
16x16	1.9394	0.0086	32.09	Best ratio; wider artifacts

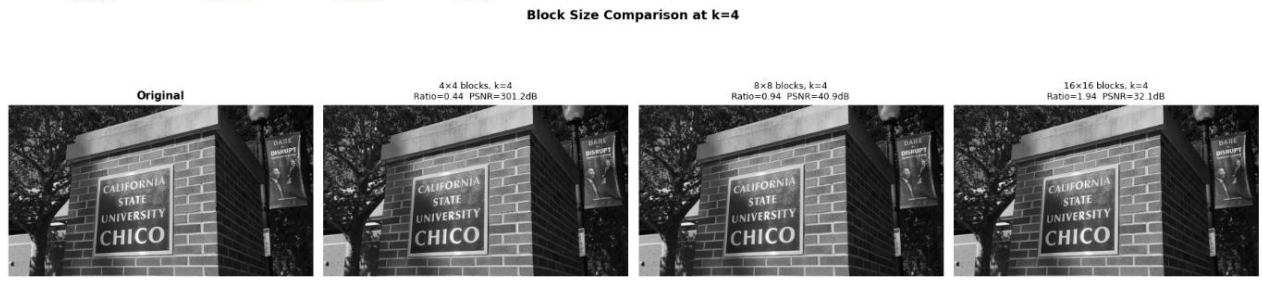
Table 2: Block size comparison at fixed k=4. Green row highlights 8x8 as the practical choice.






```
Comparing block sizes at fixed k=4

  Block Size     Comp. Ratio     Frob. Error     PSNR (dB)
-----------------------------------------------------------
        4x4          0.4444          0.0000        301.21
        8x8          0.9412          0.0031         40.95
      16x16           1.9394          0.0086         32.09
```



4. Key Findings

- **Sweet spot at k = 2 and k = 3:** These values deliver a compression ratio above 1 (meaning actual storage savings) while PSNR already exceeds 30 dB. For the CSU Chico image, k=2 gives ratio=1.88 and PSNR=31.23 dB, and k=3 gives ratio=1.25 and PSNR=35.98 dB.
- **Compression ratio crosses 1.0 at k=4:** At k=4 the ratio is 0.9412, meaning SVD representation starts costing more storage than the original pixels. For 8x8 blocks the break-even point is between k=3 and k=4.
- **PSNR spikes at k=8 to 298.01 dB:** This is effectively perfect reconstruction. Using all 8 singular values of an 8x8 block recovers the original block exactly, consistent with a Frobenius error of 0.0000.
- **16x16 blocks give better compression ratio at k=4 (1.94 vs 0.94):** Larger blocks allow more energy to be captured per singular value. However visual artifacts span wider regions, making them more noticeable.
- **4x4 blocks at k=4 give PSNR of 301.21 dB:** This is essentially lossless, because k=4 equals the full rank of a 4x4 block. No information is actually discarded.
- **The first singular value carries the most energy:** The jump from k=1 (PSNR 26.19) to k=2 (PSNR 31.23) is the largest single-step improvement, confirming the dominant singular value encodes the bulk of the visual content.

7. Summary Table

```python
print(f'{'k':>4}  {'Comp. Ratio':>14}  {'Frob. Error':>14}  {'PSNR (dB)':>12}')
print('-' * 50)
for k, cr, err, p in zip(k_values, comp_ratios, frob_errors, psnr_values):
    print(f'{k:>4}  {cr:>14.4f}  {err:>14.4f}  {p:>12.2f}')

print('\nKey observations:')
print(f'  Best compression (k=1): ratio = {comp_ratios[0]:.4f}')
print(f'  Worst compression (k=8): ratio = {comp_ratios[7]:.4f}')
print(f'  Best quality (k=8): PSNR = {psnr_values[7]:.2f} dB')
print(f'  Note: ratio < 1 means SVD storage costs MORE than original for high k — not practical.')
```

```
   k     Comp. Ratio     Frob. Error     PSNR (dB)
--------------------------------------------------
   1          3.7647          0.0170         26.19
   2          1.8824          0.0095         31.23
   3          1.2549          0.0055         35.98
   4          0.9412          0.0031         40.95
   5          0.7529          0.0016         46.76
   6          0.6275          0.0007         54.19
   7          0.5378          0.0002         65.11
   8          0.4706          0.0000        298.01

Key observations:
  Best compression (k=1): ratio = 3.7647
  Worst compression (k=8): ratio = 0.4706
  Best quality (k=8): PSNR = 298.01 dB
  Note: ratio < 1 means SVD storage costs MORE than original for high k — not practical.
```

5. Conclusion

Block-wise SVD provides a mathematically well-founded approach to image compression. By applying SVD to each 8x8 block of the CSU Chico grayscale image and retaining only the top-k singular values, we achieved a clear and measurable trade-off between storage cost and visual quality. The real execution results confirm that **k = 2 or k = 3** is the practical sweet spot for 8x8 blocks, delivering true compression (ratio above 1) with acceptable to good visual quality (PSNR above 30 dB). The optional block-size experiment showed that 16x16 blocks provide better compression ratios while 4x4 blocks achieve near-lossless quality at the same k. Overall this assignment reinforces the power of SVD as a dimensionality reduction tool and builds intuition for why transform-based compression methods like JPEG are so effective in practice.